

SQL CHEATSHEET

Pattern	Typical Question	Example SQL
Previous / Next	Compare current vs previous order	<pre>SELECT user_id, order_date, LAG(order_date) OVER (PARTITION BY user_id ORDER BY order_date) prev_order FROM orders;</pre>
Consecutive days	Find login streaks	<pre>SELECT user_id, login_date FROM (SELECT user_id, login_date - ROW_NUMBER() OVER (PARTITION BY user_id ORDER BY login_date) grp FROM logins) t GROUP BY user_id, grp;</pre>
Time gap / delay	Days between orders	<pre>SELECT user_id, DATEDIFF(order_date, LAG(order_date) OVER (PARTITION BY user_id ORDER BY order_date)) AS gap_days FROM orders;</pre>
Session / inactivity	New session after 30 mins	<pre>SELECT *, CASE WHEN TIMESTAMPDIFF(MINUTE, LAG(event_time) OVER (PARTITION BY user_id ORDER BY event_time), event_time) > 30 THEN 1 ELSE 0 END AS new_session FROM events;</pre>
Top N per group	Top 2 salaries per dept	<pre>SELECT * FROM (SELECT *, ROW_NUMBER() OVER (PARTITION BY dept ORDER BY salary DESC) rn FROM emp) t WHERE rn <= 2;</pre>

Pattern	Typical Question	Example SQL
Funnel / drop-off	Signup → Purchase	<pre>SELECT user_id, MAX(CASE WHEN event = 'signup' THEN 1 ELSE 0 END) AS signup, MAX(CASE WHEN event = 'purchase' THEN 1 ELSE 0 END) AS purchase FROM events GROUP BY user_id;</pre>
Spike / anomaly	Sudden jump in sales	<pre>SELECT day, (sales - LAG(sales) OVER (ORDER BY day)) / LAG(sales) OVER (ORDER BY day) AS pct_change FROM daily_sales;</pre>
Running total	Cumulative revenue	<pre>SELECT day, SUM(revenue) OVER (ORDER BY day ROWS BETWEEN UNBOUNDED PRECEDING AND CURRENT ROW) AS running_total FROM sales;</pre>
Conditional aggregation	Count by condition	<pre>COUNT(CASE WHEN status='completed' THEN 1 END)</pre>
Deduplication	Latest record per user	<pre>ROW_NUMBER() OVER (PARTITION BY user_id ORDER BY updated_at DESC)</pre>
Nth highest value	2nd / 3rd highest salary	<pre>DENSE_RANK() OVER (ORDER BY salary DESC)</pre>
Percent of total	Category contribution	<pre>SUM(x)/SUM(SUM(x)) OVER()</pre>
Existence check	Users with at least one event	<pre>EXISTS (SELECT 1 FROM t2 WHERE ...)</pre>



CORE PATTERN MAP

When you see in question...	Immediately think...	Key SQL Tool	Mental Trigger
previous / next	Compare rows	LAG / LEAD	Time shift
consecutive days	Streak logic	ROW_NUMBER	Date - row_num
time gap / delay	Time difference	DATEDIFF / TIMESTAMPDIFF	Lag + diff
session / inactivity	New session flag	Lag + CASE + SUM	Breakpoint
top N per group	Ranking	RANK / ROW_NUMBER	Partition
retention / churn	Cohort analysis	Month diff	Signup anchor
funnel / drop-off	Event pivot	MAX(CASE WHEN...)	NULL check
spike / anomaly	Compare periods	LAG + % change	Threshold
repeat users	Frequency	COUNT(*) > 1	Loyalty
running total	Accumulation	SUM() OVER()	Cumulative



TIME-BASED PROBLEMS

Question	Correct mental model	Example SQL
Days between orders	LAG(order_date)	DATEDIFF(order_date, LAG(order_date) OVER (PARTITION BY user_id ORDER BY order_date))
Inactive for 30 days	DATEDIFF(today, last_event)	HAVING DATEDIFF(CURDATE(), MAX(event_date)) > 30
Rolling 7 days sum	Window frame	SUM(sales) OVER (ORDER BY day ROWS BETWEEN 6 PRECEDING AND CURRENT ROW)
Month-1 retention	TIMESTAMPDIFF(MONTH, signup, activity)	TIMESTAMPDIFF(MONTH, signup_date, activity_date) = 1
Session after 30 mins	Gap > threshold	CASE WHEN TIMESTAMPDIFF(MINUTE, prev_time,event_time) > 30 THEN 1 END AS new_session

🧩 WINDOW FUNCTION QUICK DECISION TABLE

Need	Use	Why	Example
Previous row	LAG()	Compare timeline	sql LAG(amount) OVER (PARTITION BY user_id ORDER BY date)
Next row	LEAD()	Look ahead	sql LEAD(amount) OVER (PARTITION BY user_id ORDER BY date)
Unique rank	ROW_NUMBER()	No ties	sql ROW_NUMBER() OVER (ORDER BY score DESC)
Rank with ties	RANK()	Allow ties	sql RANK() OVER (ORDER BY score DESC)
Rank without gaps	DENSE_RANK()	Continuous ranks	sql DENSE_RANK() OVER (ORDER BY score DESC)
Running total	SUM() OVER()	Cumulative metric	sql SUM(amount) OVER (ORDER BY date)

📊 COHORT ANALYSIS MINI-MAP

Component	Meaning
cohort_month	WHO (signup batch)
months_since_signup	WHEN (time passed)
COUNT(DISTINCT user_id)	Active users
Retention %	Active / cohort size

🔄 FUNNEL ANALYSIS CHEAT LOGIC

Step	Pattern
Convert events → columns	MAX(CASE WHEN ...)
Missing step	Column is NULL
Drop-off detection	CASE WHEN
Time between steps	TIMESTAMPDIFF

customer_id	customer_name	city	signup_date
1	Amit Sharma	Delhi	10/1/2022
2	Anjali Verma	Mumbai	15/3/2022
3	Rahul Mehta	Bangalore	20/11/2021
4	Sneha Iyer	Chennai	5/2/2023
5	Akash Singh	Delhi	18/6/2023
6	Priya Nair	Kochi	25/9/2022
7	Rohan Das	Kolkata	12/8/2021
8	Aditi Rao	Mumbai	2/1/2023

payment_id	order_id	payment_date	payment_method	amount
1001	101	2/2/2022	Credit Card	1200
1002	102	11/5/2022	UPI	800
1003	104	21/1/2023	Debit Card	700
1004	105	2/12/2021	Net Banking	3000
1005	106	23/7/2022	Credit Card	1500
1006	107	11/2/2023	UPI	2000
1007	109	6/10/2022	Debit Card	900
1008	110	13/3/2023	Credit Card	1100
1009	111	11/1/2023	Net Banking	5000

order_id	customer_id	order_date	order_amount	status
101	1	1/2/2022	1200	Completed
102	1	10/5/2022	800	Completed
103	2	15/6/2022	450	Cancelled
104	2	20/1/2023	700	Completed
105	3	1/12/2021	3000	Completed
106	3	22/7/2022	1500	Completed
107	4	10/2/2023	2000	Completed
108	5	1/7/2023	600	Pending
109	6	5/10/2022	900	Completed
110	6	12/3/2023	1100	Completed
111	8	10/1/2023	5000	Completed

First order in 2023

```
SELECT customer_id
FROM orders
GROUP BY customer_id
HAVING MIN(order_date) BETWEEN '2023-01-01' AND '2023-12-31';
```

Rank orders per customer

```
SELECT *,
RANK() OVER (
    PARTITION BY customer_id
    ORDER BY order_amount DESC)
AS rank_no
FROM orders;
```

Days between signup & first order

```
SELECT c.customer_id,
    MIN(o.order_date) - c.signup_date AS days_gap
FROM customers c
JOIN orders o ON c.customer_id = o.customer_id
GROUP BY c.customer_id, c.signup_date;
```

Latest order per customer

```
SELECT *
FROM (
    SELECT *,
    ROW_NUMBER() OVER (
        PARTITION BY customer_id
        ORDER BY order_date DESC) AS rn
    FROM orders
) t
WHERE rn = 1;
```

Running total

```
SELECT *,
SUM(order_amount) OVER (
    PARTITION BY customer_id
    ORDER BY order_date
)
AS running_total
FROM orders;
```

Compare with previous order

```
SELECT *,
LAG(order_amount) OVER (
    PARTITION BY customer_id
    ORDER BY order_date
) AS prev_order_amount
FROM orders;
```

Second highest order

```
SELECT *
FROM (
    SELECT *,
    DENSE_RANK() OVER (
        PARTITION BY customer_id
        ORDER BY order_amount DESC
    ) AS rnk
    FROM orders
) t
WHERE rnk = 2;
```


Orders above city average

```
SELECT *
FROM (
  SELECT o.*, c.city,
  AVG(order_amount) OVER (PARTITION BY city) AS city_avg
  FROM orders o
  JOIN customers c ON o.customer_id = c.customer_id
) t
WHERE order_amount > city_avg;
```

Percent contribution

```
SELECT *,
(order_amount * 100.0) /
SUM(order_amount) OVER (
  PARTITION BY customer_id)
AS percent_contribution
FROM orders;
```

Top 3 customers by revenue

```
SELECT customer_id, SUM(order_amount) AS revenue
FROM orders
GROUP BY customer_id
ORDER BY revenue DESC
LIMIT 3;
```

Customer category

```
SELECT customer_id,
CASE
  WHEN SUM(order_amount) > 10000 THEN 'Platinum'
  WHEN SUM(order_amount) BETWEEN 5000 AND 10000 THEN 'Gold'
  ELSE 'Silver'
END AS category
FROM orders
GROUP BY customer_id;
```